

S.R.M. INSTITUTE OF SCIENCE AND TECHNOLOGY

FACULTY OF ENGINEERING AND TECHNOLOGY

SMS SPAM DETECTION USING MACHINE LEARNING

Submitted by

SANJITH S [RA2411026050214]

JOE ANDREW S [RA2411026050215]

in partial fulfillment for the award of the degree

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING WITH SPECIALIZATION IN ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

FACULTY OF ENGINEERING AND TECHNOLOGY

S.R.M. Nagar, Trichy-Chennai Highway, Tiruchirappalli-62110

OCTOBER 2026

1. Abstract

This project evaluates machine-learning methods for classifying SMS messages as spam or ham. The workflow includes text cleaning, TF-IDF feature extraction, and comparison of Decision Tree, K-Nearest Neighbors (KNN), Linear Support Vector Machine (SVM), and Multinomial Naive Bayes. The recorded results show that SVM achieved the highest accuracy (98.26%) and F1-score (0.9274) among the four tested classifiers.

2. Introduction

SMS spam messages can contain unwanted advertisements, scams, or misleading links. Automatically identifying spam can help users filter unwanted messages. This project applies supervised machine learning to classify SMS text into two categories: ham (legitimate message) and spam.

3. Dataset

The notebook reports 5,572 messages in the original dataset: 4,825 ham messages and 747 spam messages. These counts represent the dataset distribution before missing-value and duplicate-row cleaning. The notebook then removes missing values and duplicate rows before training the models.

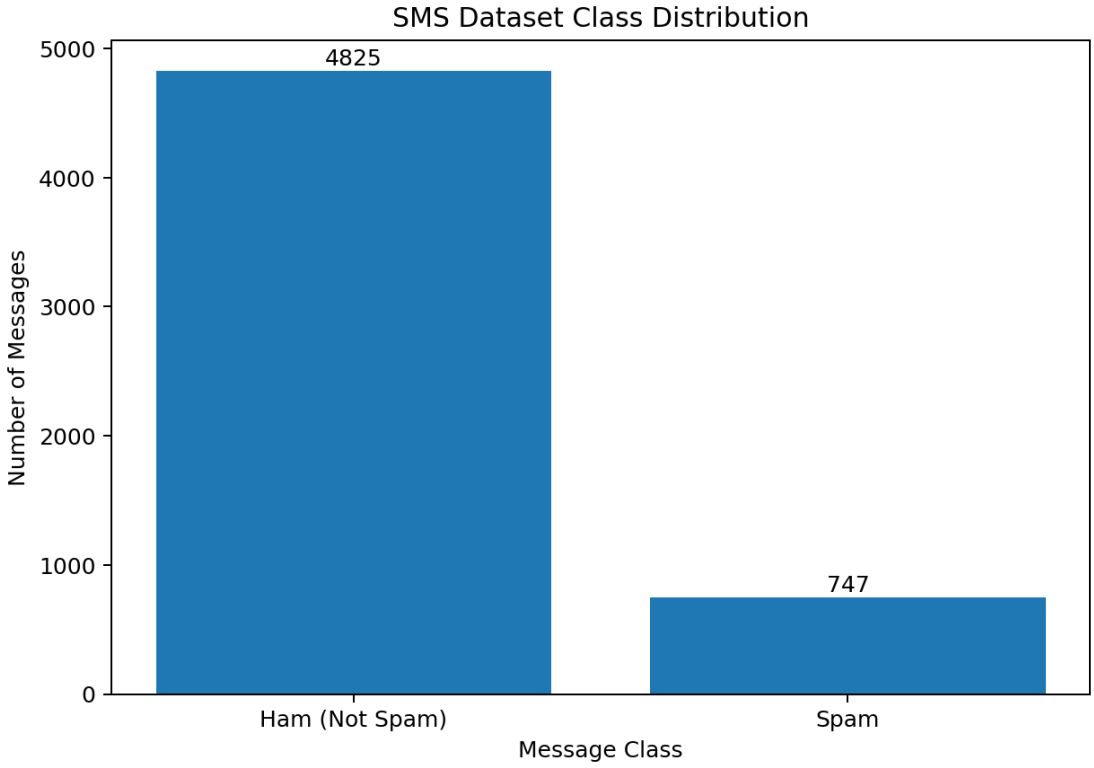


Figure 1. Class distribution of SMS messages

4. Methodology

Preprocessing: missing rows and duplicate rows are removed; message text is lowercased; URLs and digit sequences are replaced with placeholders; non-letter characters are removed; and whitespace is normalized.

Feature extraction: TF-IDF vectorization is used with max_features=5,000 and ngram_range=(1, 2).

Data split: the notebook uses a stratified 80% training and 20% testing split with random_state=42. The recorded output reports 4,135 training samples and 1,034 test samples.

Models: Decision Tree, KNN (K=5 for the main comparison), LinearSVC (SVM), and Multinomial Naive Bayes.

Evaluation: accuracy, spam-class precision, spam-class recall, F1-score, and measured runtime are compared.

5. Model Evaluation Results

Table 1 reports the actual values in the model_comparison_results.csv file. Precision and recall are important alongside accuracy because missing spam messages and incorrectly flagging legitimate messages have different consequences.

Model	Accuracy	Precision	Recall	F1-score	Runtime (s)
Decision Tree	0.9594	0.8397	0.8397	0.8397	0.7272
KNN	0.9159	1.0000	0.3359	0.5029	0.2787
SVM	0.9826	0.9829	0.8779	0.9274	0.0219
Naive Bayes	0.9691	1.0000	0.7557	0.8609	0.0304

Table 1. Comparison of model performance

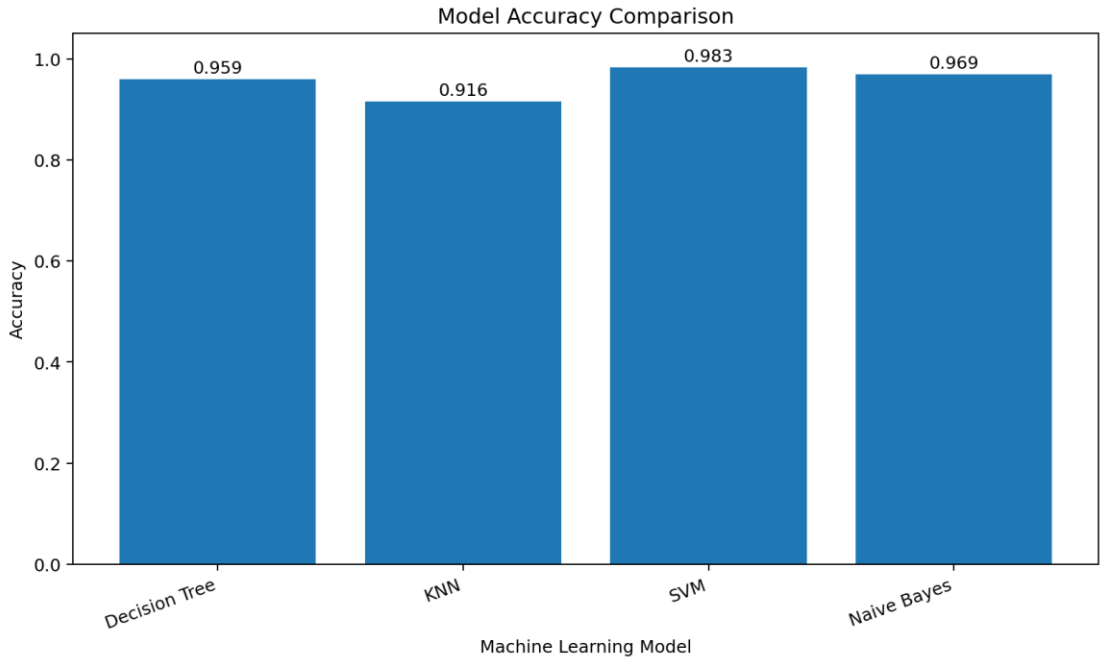


Figure 2. Model accuracy comparison

SVM achieved the highest accuracy (0.9826) and F1-score (0.9274). Naive Bayes achieved precision of 1.0000 but had lower recall (0.7557). KNN at K=5 had recall of 0.3359. These results show why accuracy alone should not determine the preferred model.

6. KNN What-If Experiment

The KNN experiment compares K values of 3, 5, 7, 9, and 11. The values below are taken from knn_what_if_results.csv.

K	Accuracy	Precision	Recall	F1-score
3	0.9284	1.0000	0.4351	0.6064
5	0.9159	1.0000	0.3359	0.5029
7	0.8994	1.0000	0.2061	0.3418
9	0.8956	1.0000	0.1756	0.2987
11	0.8917	1.0000	0.1450	0.2533

Table 2. KNN performance for different K values



Figure 3. KNN accuracy across K values

Among the tested K values, K=3 had the highest accuracy (0.9284) and F1-score (0.6064). Accuracy and F1-score declined as K increased in this experiment.

7. Conclusion

The recorded experiment identifies Linear SVM as the strongest overall model among the four tested classifiers, with 98.26% accuracy and an F1-score of 0.9274. The KNN experiment shows that larger K values did not improve performance in this setup. Future work could include cross-validation, further feature engineering, and review of confusion matrices to understand false positives and false negatives.

8. Project Files

- SMS_Spam_Classification.ipynb — implementation and experiments
- README.md — project overview and usage notes
- model_comparison_results.csv — model evaluation metrics
- knn_what_if_results.csv — KNN experiment results
- class_distribution.png — class distribution graph
- model_comparison.png — model comparison graph
- knn_what_if.png — KNN what-if graph